ASSIGNMENT – 2

DEEP LEARNING

Name	Kamalesh T
Registration Number	192424228
Course Code	CSA47
Course	Deep Learning
CO2	Deep Feed-Forward and Convolutional Network Design

TASK 1 – Deep Feed-Forward Network Architecture (5 Marks)

(a) The input layer has 4 neurons for rainfall, temperature, soil moisture and fertilizer quantity. The first hidden layer has 8 neurons, the second hidden layer has 4 neurons, and the output layer has 1 neuron for predicted crop yield. Architecture: $4 \rightarrow 8 \rightarrow 4 \rightarrow 1$.

Layer	Neurons	Function
Input	4	Receives crop measurements
Hidden layer 1	8	Learns feature combinations
Hidden layer 2	4	Learns higher-level patterns
Output	1	Predicts crop yield

(b) Using column-vector notation: W1 has dimensions 8×4 and b1 has 8 elements; W2 is 4×8 and b2 has 4 elements; W3 is 1×4 and b3 has 1 element.

(c) Forward propagation: $a0=x$; $a1=f(W1a0+b1)$; $a2=f(W2a1+b2)$; and $\hat{y}=W3a2+b3$. The hidden activation function f introduces non-linearity. A linear output is appropriate for continuous crop-yield prediction.

TASK 2 – Forward Propagation and Activation Functions (2 Marks)

(a) Weighted sum $z = (0.6\times0.4)+(0.8\times0.3)+(0.5\times0.2)+0.1 = 0.24+0.24+0.10+0.10 = 0.68$.

(b) $\text{ReLU}(z)=\max(0,z)=\max(0,0.68)=0.68$. Since the weighted sum is positive, ReLU keeps it unchanged.

TASK 3 – Gradient-Based Learning and Loss Calculation (6 Marks)

(a) A loss function measures the difference between predicted and actual values and provides an objective for training. For continuous crop-yield prediction, squared error or mean squared error (MSE) is suitable.

(b) Actual $y=50$ and predicted $\hat{y}=42$. $L=(y-\hat{y})^2=(50-42)^2=8^2=64$.

(c) The squared error is 64 squared-yield units. The prediction is 8 units below the actual yield. Zero loss would indicate an exact prediction for this example.

(d) Gradient-based learning calculates derivatives of the loss with respect to each weight and bias. The optimizer updates parameters in a direction intended to reduce the loss. This is repeated over training batches and epochs.

TASK 4 – Gradient Descent and Weight Update (4 Marks)

(a) Update rule: $w_{new}=w_{old}-\eta(\partial L/\partial w)$. Thus $w_{new}=0.5-(0.1\times0.8)=0.42$.

(b) The gradient points toward the steepest local increase in loss. Moving in the opposite direction is intended to reduce the loss.

(c) A very small learning rate makes training slow. A very large rate may overshoot a minimum, cause oscillation or divergence. A suitable rate balances speed and stability.

TASK 5 – Back-Propagation Through Multiple Layers (3 Marks)

(a) Backpropagation determines how much each weight and bias contributes to the final prediction error by propagating error information backward from the output layer.

(b) The chain rule calculates gradients through successive layers. It multiplies the local derivatives along the path from an early-layer parameter to the final loss.

(c) The calculated gradients are used by an optimizer to update weights and biases, commonly with $w_new = w_old - \eta \nabla w$.

TASK 6 – Activation Functions and Vanishing/Exploding Gradients (3 Marks)

(a) Sigmoid and tanh saturate for large positive or negative inputs. Their derivatives become very small in saturated regions; multiplying many small derivatives during backpropagation can cause vanishing gradients. Very large derivative products can cause exploding gradients.

Activation	Range	Hidden-layer characteristics
Sigmoid	0 to 1	Smooth but saturates; can cause vanishing gradients and is not zero-centred.
Tanh	−1 to 1	Zero-centred but also saturates.
ReLU	0 to +∞	Simple gradient for positive inputs; often supports faster deep-network training, but inactive units can have zero gradient.

(b) Sigmoid is often avoided in deep hidden layers because of saturation. Tanh is zero-centred but can also saturate. ReLU is commonly used in hidden layers because positive inputs do not saturate.

(c) Activation choice affects gradient flow, convergence and learned representations. Use a suitable hidden activation such as ReLU or a variant, and choose the output activation according to the task.

TASK 7 – Back-Propagation and Optimization Strategy (4 Marks)

(a) Learning rate controls the size of parameter updates. A small rate can make convergence slow; a large rate can cause unstable or fluctuating loss.

(b) One iteration performs forward propagation, calculates loss, uses backpropagation to calculate gradients, and then applies gradient descent or another optimizer to update parameters.

(c) If loss decreases slowly, try a carefully increased learning rate, Adam, better scaling or a learning-rate schedule. If loss fluctuates, reduce the learning rate, adjust mini-batch size, use momentum/Adam and check the data and gradients.

(d) The loss curve indicates progress, stability or divergence. Comparing training and validation curves helps detect overfitting and choose when to stop training.

TASK 8 – Regularization and Overfitting Control (4 Marks)

(a) The model is likely overfitting: training accuracy is 99%, while validation accuracy is only 78%, showing much poorer performance on unseen examples.

(b) L1 adds the sum of absolute weights to the loss and can encourage sparse weights. L2 adds squared weights and discourages excessively large parameter values.

(c) $L_{total} = L_{data} + \lambda \Sigma w_i^2$ (sometimes with a factor of 1/2). L_{data} is the original task loss; λ controls the strength of the penalty. Too little regularization may not control overfitting, while too much may cause underfitting.

(d) Dropout randomly sets selected activations to zero during training. It reduces reliance on particular neurons and encourages robust representations. It is disabled during inference.

TASK 9 – Convolution Operation and Feature Extraction (3 Marks)

(a) Convolutional layers suit images because local receptive fields and shared filters detect spatial patterns such as edges, spots and textures with fewer parameters than fully connected processing of every pixel.

(b) Example: image = [[1,2,0],[0,1,3],[2,1,0]], filter = [[1,0],[0,−1]], valid convolution, stride 1. The four 2×2 patches produce: top-left 1−1=0; top-right 2−3=−1; bottom-left 0−1=−1; bottom-right 1−0=1. Feature map = [[0,−1],[−1,1]]. It represents the filter response at each local position.

(c) Filters are learnable kernels that detect patterns. Stride is the step size as a filter moves and affects output dimensions. Padding adds border values (often zeros) to control output size and preserve border information.

TASK 10 – CNN Architecture and Pooling (4 Marks)

(a) Input image supplies pixels. Convolution extracts local features. ReLU adds non-linearity. Pooling reduces spatial size. Further convolution layers learn more complex features. The fully connected layer combines extracted features, and the output layer predicts healthy or diseased classes.

(b) Example of 2×2 max pooling with stride 2: input [[1,3,2,4],[5,6,1,2],[0,2,7,3],[1,4,2,8]] becomes [[6,4],[4,8]]. Each output is the maximum in a 2×2 region, reducing dimensions from 4×4 to 2×2.

(c) Filter size controls the local region examined. Stride controls movement and output size. Number of filters controls output channels and the number of learned feature types; more filters increase computation.

(d) Early layers learn simple edges and colour transitions. Deeper layers combine them into textures, shapes and larger structures, potentially identifying disease-related leaf patterns.

TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models (4 Marks)

Aspect	Model A: Feed-forward	Model B: CNN
Input	Fixed-length numerical feature vector	Image pixels and channels
Feature extraction	Dense layers combine numerical features	Convolution filters learn spatial features
Parameters	Dense connections between layers	Shared convolution filters plus dense output layers
Computation	Often efficient for a few numerical features	Depends on image size, channels, filters and depth

(a) Model A suits rainfall, temperature, soil moisture and fertilizer quantity because these are structured numerical features.

(b) Model B suits leaf images because convolution uses spatial structure and learns visual patterns such as spots, edges and texture.

(c) A dense network connects neurons across adjacent layers. A CNN uses local connections and shared filters, which exploit image structure and can reduce parameter count compared with fully connected image processing.

(d) Recommend a feed-forward network for numerical crop-yield prediction and a CNN for healthy/diseased leaf-image classification. Each model matches the structure of its input data.

TASK 12 – Design Challenge: Optimize and Redesign the Deep Learning System (8 Marks)

A. Impact Analysis (2 Marks)

(a) Increasing numerical records from 10,000 to 500,000 raises storage, preprocessing, memory and training requirements. Adding 100,000 leaf images introduces high-dimensional image data and requires image preprocessing and a vision architecture. Mini-batch training and efficient data loading can improve throughput.

(b) A feed-forward network applied directly to flattened images loses explicit two-dimensional spatial structure and may require many parameters. It does not naturally exploit local patterns or shared filters, making it less efficient for image feature extraction.

B. Evaluation of the Existing Design (2 Marks)

(a) High training accuracy with lower validation performance is evidence of overfitting and weak generalization. Long training time also indicates a need to investigate efficiency. Training and validation loss curves should be examined.

(b) Simply increasing depth and neuron count is not necessarily best. It increases capacity and computation and may worsen overfitting. Architecture should match the data type, with regularization and validation guiding model selection.

C. Adaptation or Redesign (2 Marks)

(a) Use a feed-forward network for numerical crop data, with four input features and one linear yield output. Use a separate CNN for leaf images, with convolution, ReLU, pooling and a final classifier for healthy/diseased categories. Keep preprocessing and evaluation pipelines appropriate to each task.

(b) Adam offers adaptive parameter updates; compare it with SGD. Mini-batch SGD processes subsets of records; momentum can smooth updates; RMSProp adapts update sizes. Start with a validated learning rate and reduce it when validation loss plateaus. Use dropout and L2 regularization to control overfitting, batch normalization to support stable training, ReLU or a variant in hidden layers, linear output for yield regression and sigmoid/softmax for classification.

Image augmentation should preserve labels: use suitable horizontal/vertical flips, small rotations, modest crops and mild brightness/contrast changes only when these do not change the disease label.

Model comparison: LeNet is a compact baseline; AlexNet is deeper and more computationally demanding; ResNet uses skip connections to support deep training; EfficientNet targets an accuracy-efficiency balance. Select using image resolution, validation performance and available compute budget.

D. Evidence-Based Defence (2 Marks)

(a) Compare the original and redesigned systems using fixed train/validation/test splits. For yield regression, report MSE or RMSE and MAE. For leaf classification, report accuracy, precision, recall, F1-score, confusion matrix and, where appropriate, ROC-AUC. Record training time, inference time and memory use. Prevent data leakage between splits.

(b) The expected evidence supports redesigning into task-specific models: a feed-forward network for numerical crop data and a CNN for leaf images. Address the training-validation gap with suitable capacity, dropout, L2, augmentation and early stopping. Retain an original component only if controlled testing shows it is accurate and efficient.

CONCLUSION

Deep feed-forward networks are suitable for structured numerical crop data, while CNNs learn spatial features from leaf images. Forward propagation produces predictions, backpropagation computes gradients, and optimization updates parameters to reduce loss. Appropriate activations, learning rates and regularization improve training and generalization. For the expanded smart-agriculture requirement, separate task-specific models with evidence-based evaluation provide a suitable design.